FleetWorks Application Architecture Design

Updated on: 2026-09-06

1. Executive Summary

FleetWorks is a static, browser-first fleet management application backed by Supabase. The application is built with multi-page HTML, shared CSS, and vanilla JavaScript modules that run directly in the browser. Supabase provides the production backend capabilities: authentication, Postgres database, Row Level Security, REST/RPC APIs, private file storage, and Edge Functions.

The current architecture is best described as:

> Static multi-page SaaS and PWA with a browser modular monolith, hexagonal boundaries inside business modules, and Supabase as the backend platform.

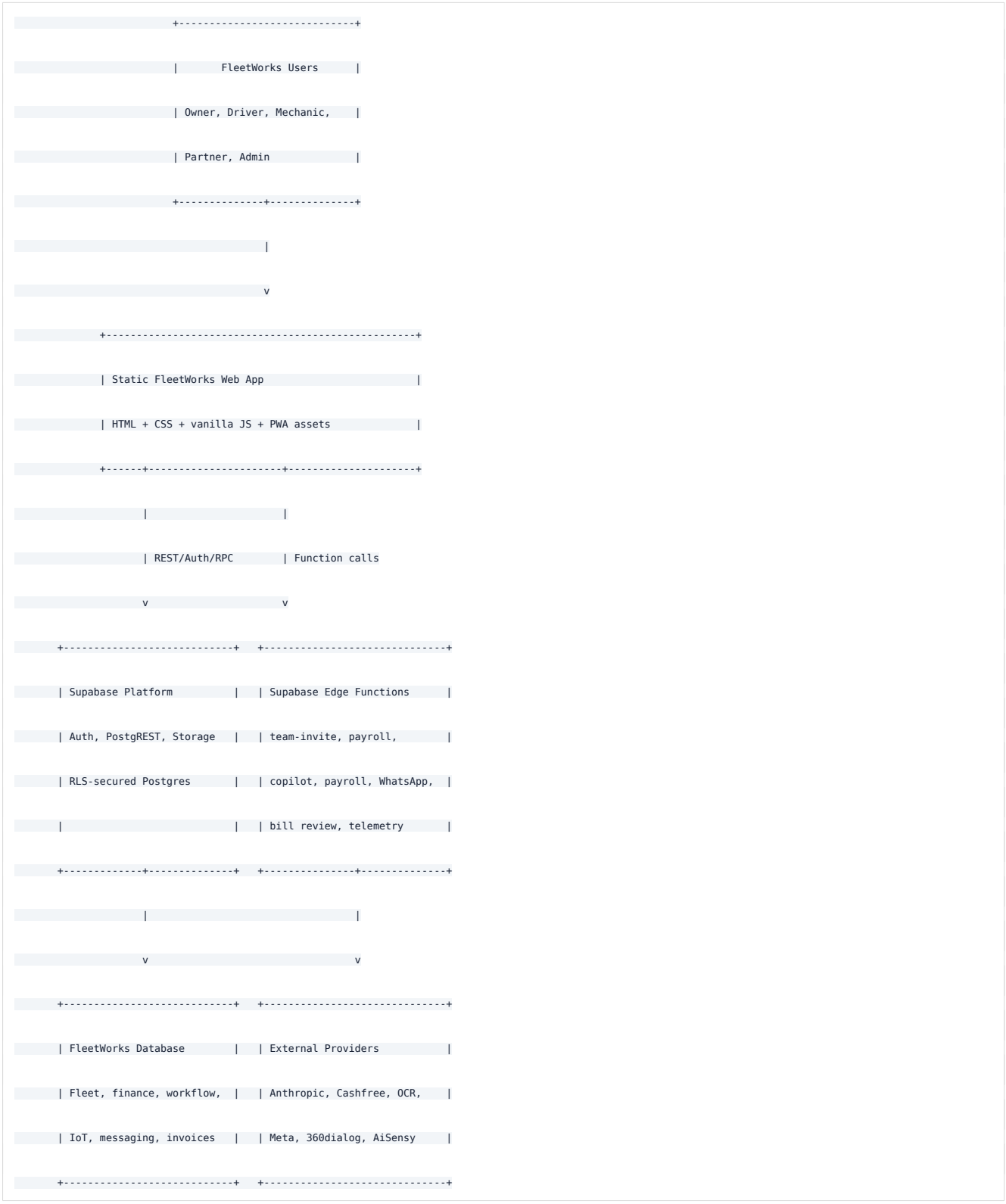
There is no traditional application server for business logic. The local Node server only serves static files for development and testing.

2. Layer-Wise System Architecture


| FWApi Stable data and Edge Function facade |
| dbcore.js DB-direct mapping retained as transitional infrastructure |
| cloudstore.js Auth, session, REST, RPC, sync, and storage infrastructure |
+-----------------------------------+-----------------------------------+
|
v
+---+
| CLIENT STATE AND OFFLINE LAYER |
| |
| localStorage keys: |
| ff_fleet Local/demo fleet state and settings |
| ff_fleet:<user> Per-user cached fleet blob |
| fw_session:<user> Supabase auth session cache |
| fw_service_requests Local playable service workflow state |
| |
| Signed-out mode: local/demo state only |
| Signed-in mode: core entities are DB-direct; settings remain blob-backed |
+-----------------------------------+-----------------------------------+
|
v
+---+
| SUPABASE API LAYER |
| |
| Auth API Email/password login, signup, recovery, token refresh |
| PostgREST Table CRUD and RPC calls from browser modules |
| Storage API Private bill/document uploads and signed URLs |
| Edge Functions Secret-bearing or privileged server-side operations |
+-----------------------------------+-----------------------------------+
|
v
+---+


DATA, SECURITY, AND DOMAIN LAYER
Postgres schema with RLS:
Tenant core: organizations, memberships
Fleet core: vehicles, drivers, expenses, fuel_logs, issues, work_orders
Operations: reminders, inspections, parts, documents, tyre_readings
Finance: trips, ledger, payroll, FASTag, parties, items, sales invoices
Team access: vehicle_assignments, expense_change_requests
Intelligence: bill_reviews, work_order_lines, model-backed risk scoring
Communications: WhatsApp contacts, messages, templates, attendance
Telemetry: devices, telemetry, ADAS events, retention policies
Intake: leads, vendors, insurance quotes, driver operational entries
Service workflow: service_requests, assignments, estimates, invoices, payments
Main security boundary: Supabase Row Level Security policies
EXTERNAL SERVICE LAYER
Anthropic Claude
Copilot Edge Function
Cashfree Payouts
Payroll Edge Functions
Meta / 360dialog / AiSensy
WhatsApp Edge Functions
PaddleOCR/FastAPI
Optional bill OCR microservice
FASTag / UPI
Owner-directed recharge actions

3. Typical System Context Diagram



4. Runtime and Deployment Architecture

The local server in `server/static-server.mjs` only maps URL paths to files and returns them with the right MIME type. Production can be hosted on any static hosting platform as long as the Supabase project remains configured.

5. Main Application Modules

Current code is split into two layers:

- `js/modules/*` owns migrated business workflows, domain rules, ports, adapters, use cases, view-model helpers, and module-owned page controllers.
- `js/legacy/*` contains deprecated compatibility markers for UI controller paths that have moved.

New business logic should be added under `js/modules/*`. UI controllers should call module APIs first and keep inline behavior only as a temporary fallback while larger controllers are split.

5.1 Owner Fleet Manager

Primary page: `fleet.html`

Business modules loaded by this page:

- `js/modules/auth/*`
- `js/modules/fleet-core/*`
- `js/modules/fleet-ops/*`
- `js/modules/maintenance/*`
- `js/modules/fleet-fin/*`
- `js/modules/fleet-iq/*`
- `js/modules/team-access/*`
- `js/modules/bulk-import/*`
- `js/modules/service-workflow/*`
- `js/modules/payments/*`
- `js/modules/driver-map/*`
- `js/modules/llm-gateway/*`
- `js/modules/security/*`
- `js/modules/iot-telemetry/*`
- `js/modules/communications/*`
- `js/modules/invoicing/*`

UI controller scripts:

- `js/modules/fleet-ops/controllers/fleet.controller.js`
- `js/dbcore.js`
- `js/cloudstore.js`

- `js/modules/fleet-iq/controllers/analytics.controller.js`
- `js/modules/fleet-iq/controllers/dashboard.controller.js`
- `js/modules/fleet-iq/xgboost.js`
- `js/modules/maintenance/controllers/bill-review.controller.js`
- `js/modules/communications/controllers/whatsapp.controller.js`
- `js/modules/invoicing/controllers/invoices.controller.js`
- `js/account.js`
- `js/modules/payments/controllers/payroll.controller.js`
- `js/modules/bulk-import/controllers/bulkimport.controller.js`
- `js/modules/driver-map/controllers/fleetmap.controller.js`
- `js/modules/service-workflow/controllers/workflow.controller.js`
- `js/modules/llm-gateway/controllers/copilot.controller.js`

Responsibilities:

- Authentication gate and demo mode
- Vehicle and driver management
- Compliance tracking for insurance, PUC, fitness, permits, and road tax
- Fuel and mileage tracking
- Expense tracking and approvals
- Inspections and issue management
- Work orders and service reminders
- Parts inventory and tyre health
- Trips, loads, driver ledger, and payroll
- FleetIQ analytics, reports, and Copilot assistant
- FASTag account monitoring and recharge links
- GST-aware sales invoicing, parties, and items
- Consent-aware WhatsApp communication
- Itemized maintenance bills and AI-assisted bill review

5.2 Communications

Business module: `js/modules/communications/*`

The module declares WhatsApp contacts, messages, templates, consent permissions, and the `whatsapp-send` and `whatsapp-webhook` Edge Functions. Provider secrets remain server-side. The browser controller manages consent-aware operator flows.

5.3 Invoicing

Business module: `js/modules/invoicing/*`

The module owns parties, items, sales invoices, invoice lines, GST calculation, printing, and invoice payment status. It depends on Fleet Core and FleetFin.

5.4 Insurance

Business module: `js/modules/insurance/*`

The module owns commercial vehicle premium estimates and insurance quote requests. Shared Indian geography reference data lives in `js/shared/`.

5.5 Team Portal

Primary page: `team.html`

Business module: `js/modules/team-access/*`

Module UI controller:
`js/modules/team-access/controllers/team.controller.js`

Responsibilities:

- Supervisor/driver login via Supabase Auth
- Read only the vehicles allowed by RLS and `vehicle_assignments`
- Log fuel
- Report issues
- Submit expense change requests for owner approval
- View recent vehicle history

5.6 No-Login Driver Link

Primary page: `driver.html`

Business module: `js/modules/driver-portal/*`

Module UI controller:
`js/modules/driver-portal/controllers/driver.controller.js`

Responsibilities:

- Accept owner id, token, driver name, and vehicle name from URL query parameters
- Let a driver submit fuel, issue, or inspection entries without signing in

- Insert rows into `driver_entries` using the anonymous Supabase role
- Owner later pulls and merges entries

5.7 Garage / Mechanic Workflow

Primary page: `garage.html`

Business modules:

- `js/modules/garage-ops/*`
- `js/modules/service-workflow/*`

UI controllers:

- `js/modules/garage-ops/controllers/garage.controller.js`
- `js/modules/service-workflow/controllers/workflow.controller.js`

Responsibilities:

- Service request lifecycle from complaint to closure
- Mechanic assessment, estimate, work progress, completion report, invoice, payment, feedback
- Local playable flow via `fw_service_requests`
- Cloud twin via normalized service workflow tables

5.8 Public Intake and Admin

Primary pages:

- `index.html`
- `partner.html`
- `admin.html`
- `signin.html`

Responsibilities:

- Lead capture into `leads`
- Vendor/partner application intake
- Admin review and update flows
- Staff/admin access gated by app metadata and RLS policies

6. Data Architecture

6.1 Tenant Model

```
auth.users

    |

    v

memberships ----> organizations

    |

    +-- role: owner, manager, viewer, supervisor, driver
```

The organization is the main tenant boundary. Most operational tables include `org_id`, and access is controlled by RLS functions such as `is_org_member`, `is_org_admin`, and vehicle-specific access helpers.

6.2 Core Fleet Entity Model

```
organizations

    |

    +-- vehicles

    |     |

    |     +-- fuel_logs

    |     +-- expenses

    |     +-- issues

    |     |     |

    |     |     +-- work_orders

    |     |           +-- work_order_lines

    |     |           +-- bill_reviews

    |     |

    |     +-- reminders

    |     +-- inspections

    |     +-- tyre_readings
```

```
    |     +-- documents
    |     +-- trips
    |     +-- fastag_accounts
    |     +-- devices
    |           +-- telemetry
    |           +-- adas_events
    |
    +-- drivers
    |     |
    |     +-- documents
    |     +-- driver_ledger
    |     +-- driver_payout_details
    |
    +-- parts
    +-- memberships
    +-- vehicle_assignments
    +-- expense_categories
    +-- whatsapp_contacts
    |     +-- whatsapp_messages
    +-- parties
          +-- sales_invoices
                +-- sales_invoice_lines
```

Important ID convention:

- Vehicles, drivers, and issues keep a stable browser-era `ext_id` as local `id`.
- Their real Postgres UUID is held as `dbId` in the browser.
- Other DB-direct records use the Postgres UUID directly as their local `id`.

This preserves compatibility with older render code while moving core data into normalized tables.

6.3 Hybrid Storage Model

```
Signed out:

  Browser localStorage only



Signed in:

  Core entities:

    vehicles, drivers, expenses, fuel_logs, issues, work_orders,

    reminders, inspections, parts, documents, tyre_readings, trips,

    driver_ledger, fastag_accounts, fastag_balance_log,

    work_order_lines, bill_reviews, parties, items, sales_invoices,

    sales_invoice_lines, whatsapp_contacts, whatsapp_messages

    -> Supabase normalized tables



  Settings/demo metadata:

    -> localStorage + fleets.data blob

    -> projected into organizations by sync_fleet_from_blob
```

This design prevents stale JSON blob data from overwriting normalized direct writes.

7. Backend Components

7.1 Supabase Auth

Used for:

- Owner login/signup
- Supervisor/driver/team accounts
- Password recovery
- JWT-based access to PostgREST
- Edge Function caller verification

7.2 Supabase PostgREST

The browser calls Supabase REST endpoints directly through helpers in `cloudstore.js`:

- `authGet`
- `authInsert`
- `authInsertRet`
- `authPatch`
- `authDelete`
- `authRpc`

7.3 Supabase RLS

RLS is the primary authorization layer. The browser may request data, but the database decides what is visible or writable.

Policy groups include:

- Owner-only fleet blob access
- Organization membership access
- Vehicle assignment access for supervisors/drivers
- Expense approval workflow
- Payroll visibility and updates
- Admin access for leads and vendor applications
- Private storage object access by user folder

7.4 Supabase Storage

Private `bills` bucket stores uploaded bill images/PDFs. Object access is scoped by user id folder and signed URLs.

7.5 Edge Functions

```
team-invite
  Creates auth users, memberships, and vehicle assignments using service role.

copilot
  Holds Anthropic API key server-side and returns fleet assistant answers.

payroll-add-beneficiary
payroll-transfer
payroll-webhook
  Integrate with Cashfree payouts and record payroll activity.

owner-signup
admin-reset-password
  Perform privileged account creation and reset operations without exposing the service role.

bill-review
  Reviews itemized work-order bills against fleet history before job closure.

telemetry-ingest
  Accepts authenticated device telemetry and writes normalized records.

whatsapp-send
whatsapp-webhook
  Send consent-checked messages and process provider delivery or driver replies.
```

```
vendor-scrape

  Performs controlled server-side vendor discovery and verification.
```

8. External Integration Architecture

```
Browser
  |
  +-- Supabase Edge Function: copilot
  |       |
  |       +-- Anthropic Claude API
  |
  +-- Supabase Edge Function: payroll-*
  |       |
  |       +-- Cashfree Payouts
  |
  +-- Supabase Edge Functions: whatsapp-send / whatsapp-webhook
  |       |
  |       +-- Meta Cloud API / 360dialog / AiSensy
  |
  +-- Supabase Edge Function: telemetry-ingest
  |       |
  |       +-- Fleet devices and simulated telemetry adapters
  |
  +-- Optional OCR API: server/ocr/app.py
  |       |
  |       +-- PaddleOCR model
  |
  +-- Client-opened links
          |
          +-- WhatsApp click-to-chat
          +-- UPI payment deep links
```

9. Key Data Flows

9.1 Owner Sign-In and Boot

```
fleet.html
  -> cloudstore.js login
  -> Supabase Auth token
  -> pull fleets.data settings blob
  -> dbcore.js resolves org_id
  -> load normalized core tables
  -> FleetOps controller renderAll
```

9.2 Owner Saves a Core Record

```
Fleet form submit
  -> owning module/controller validates and builds the domain shape
  -> dbcore.js maps local shape to database row
  -> fwCloud authenticated REST helper
  -> Supabase PostgREST
  -> RLS check
  -> Postgres write
  -> local in-memory db updated
  -> render affected panels
```

9.3 Team Member Updates Vehicle Activity

```
team.html login
  -> membership lookup
  -> vehicle_assignments lookup
  -> vehicles query
  -> RLS limits returned vehicles
  -> fuel/issues writes direct when allowed
```

```
  -> expense writes become expense_change_requests

  -> owner approves/rejects in fleet.html
```

9.4 No-Login Driver Submission

```
Owner copies driver link

  -> driver.html?o=<owner>&t=<token>&n=<driver>&v=<vehicle>

  -> driver submits fuel/issue/inspection

  -> anonymous insert into driver_entries

  -> owner app later imports/merges entries
```

9.5 Copilot Question

```
js/modules/llm-gateway/controllers/copilot.controller.js

  -> creates compact fleet summary

  -> calls Supabase Edge Function copilot

  -> Edge Function calls Anthropic

  -> answer returned to browser

  -> rule-based fallback remains available
```

10. Security Architecture

Security model:

- Public anon key is intentionally present in the frontend.
- Supabase RLS protects table access.
- Authenticated browser requests use the user's JWT.
- Service role key is only inside Edge Functions.
- Sensitive integrations such as Anthropic and Cashfree run server-side.
- Storage files are private and scoped by user id path.
- Team users see only assigned vehicles through RLS.
- No-login driver links can only insert into the specific submission table.

Important security implication:

The browser is trusted for UX, not for authorization. Authorization must stay in Supabase RLS and Edge Functions.

11. Current Architectural Strengths

- Simple static hosting and low operational complexity
- Good offline/demo fallback through localStorage
- Strong Supabase-centered authorization model
- Clear multi-tenant boundary via organizations and memberships
- Normalized DB-direct path for important operational records
- Edge Functions keep privileged actions and external secrets out of the browser
- Modular feature files make the current code easy to navigate without a build step

12. Current Architectural Risks and Improvement Areas

- Browser-global module order is part of the runtime contract.
- The FleetOps controller remains large and still contains several presentation responsibilities.
- Shared mutable global `db` object can make state changes hard to reason about.
- Some state remains hybrid between normalized tables, localStorage, and blob sync.
- DOC/workflow comments show production cloud twins, but some flows are still local-first/playable.
- Direct PostgREST calls from many modules can duplicate mapping and error-handling patterns.

13. Recommended Next Architecture Evolution

Short term:

- Keep the static/Supabase architecture.
- Keep module ownership and data ownership documented per feature.
- Continue moving production records from blobs/localStorage to normalized tables.
- Keep all authorization in RLS, not client-side filters.
- Keep UI controllers under `js/modules//controllers/`; *add new business rules to* `js/modules/`.

Medium term:

- Extract a small typed data-access layer per domain.
- Split page controllers and render orchestration into smaller module-owned controllers/view models.
- Define a single app state facade around `db`.
- Add integration tests for key RLS-driven flows.

Long term:

- Consider a component framework only when UI/state complexity outweighs the simplicity of static HTML.
- Consider a small backend-for-frontend only if orchestration becomes too complex for browser-plus-Supabase.
- Add observability for Edge Functions and critical PostgREST failures.

14. Source Files Referenced

- `package.json`
- `server/static-server.mjs`
- `fleet.html`
- `team.html`
- `driver.html`
- `garage.html`
- `js/backend.js`
- `js/cloudstore.js`
- `js/dbcore.js`
- `js/modules/*`
- `js/modules/fleet-ops/controllers/fleet.controller.js`
- `js/modules/team-access/controllers/team.controller.js`
- `js/modules/driver-portal/controllers/driver.controller.js`
- `js/modules/service-workflow/controllers/workflow.controller.js`
- `js/modules/payments/controllers/payroll.controller.js`
- `js/modules/llm-gateway/controllers/copilot.controller.js`
- `js/modules/communications/controllers/whatsapp.controller.js`
- `js/modules/invoicing/controllers/invoices.controller.js`
- `js/modules/maintenance/controllers/bill-review.controller.js`
- `js/modules/fleet-iq/controllers/dashboard.controller.js`
- `supabase/migrations/*.sql`
- `supabase/functions/*/index.ts`
- `server/ocr/app.py`